

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

Hệ thống hỗ trợ tuyển dụng toàn diện EasyHR

NGUYỄN MINH HIẾU
hieu.nm224983@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính IT1

Giảng viên hướng dẫn: PGS.TS Cao Tuấn Dũng

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và truyền thông

HÀ NỘI, 05/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

Hệ thống hỗ trợ tuyển dụng toàn diện EasyHR

NGUYỄN MINH HIẾU
hieu.nm224983@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính IT1

Giảng viên hướng dẫn: PGS.TS Cao Tuấn Dũng _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và truyền thông

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn chân thành tới PGS.TS Cao Tuấn Dũng, giảng viên hướng dẫn đã định hướng, góp ý và giúp em nhìn nhận đề tài theo cách thực tế hơn trong suốt quá trình thực hiện đồ án tốt nghiệp. Những góp ý của thầy giúp em không chỉ tập trung vào việc làm cho hệ thống “chạy được”, mà còn chú ý hơn tới cách tổ chức dữ liệu, thiết kế luồng nghiệp vụ và khả năng mở rộng của sản phẩm.

Em cũng xin cảm ơn các thầy cô Trường Công nghệ thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã trang bị cho em nền tảng kiến thức về lập trình, cơ sở dữ liệu, trí tuệ nhân tạo và phát triển phần mềm. Đây là những kiến thức quan trọng để em có thể xây dựng hệ thống EasyHR trong phạm vi đồ án này.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã động viên, hỗ trợ em trong quá trình học tập và hoàn thành đồ án. Do thời gian và kinh nghiệm thực tế còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em rất mong nhận được ý kiến góp ý của thầy cô để tiếp tục hoàn thiện sản phẩm.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án này trình bày quá trình phân tích, thiết kế và xây dựng EasyHR, một hệ thống hỗ trợ tuyển dụng toàn diện dành cho nhà tuyển dụng. Hệ thống tập trung giải quyết các công việc thường tốn nhiều thời gian trong quy trình tuyển dụng như tiếp nhận CV, trích xuất thông tin ứng viên, quản lý hồ sơ, so khớp ứng viên với mô tả công việc, tạo danh sách rút gọn, hỗ trợ trao đổi với ứng viên và chuẩn bị câu hỏi phỏng vấn.

EasyHR được xây dựng theo kiến trúc web gồm giao diện người dùng, dịch vụ backend và cơ sở dữ liệu quan hệ. Phần backend sử dụng FastAPI, PostgreSQL, Redis và Celery để tổ chức các nghiệp vụ xử lý hồ sơ, chấm điểm và lưu vết dữ liệu. Phần frontend sử dụng React và Vite để xây dựng giao diện làm việc cho nhà tuyển dụng. Một số chức năng có sử dụng mô hình ngôn ngữ lớn (Large Language Model - mô hình AI có khả năng xử lý và sinh ngôn ngữ tự nhiên) nhằm hỗ trợ phân tích CV, chấm điểm mức độ phù hợp và sinh nội dung gợi ý.

Kết quả của đồ án là một nguyên mẫu hệ thống có thể chạy cục bộ bằng Docker, cung cấp các luồng nghiệp vụ chính của tuyển dụng hiện đại. Báo cáo đồng thời trình bày các hạn chế còn tồn tại, đặc biệt ở khả năng xử lý bất đồng bộ, bảo mật dữ liệu và đánh giá định lượng chất lượng gợi ý của AI.

Từ khóa: tuyển dụng, quản lý CV, trí tuệ nhân tạo, FastAPI, React, PostgreSQL, mô hình ngôn ngữ lớn.

ABSTRACT

This thesis presents the design and implementation of EasyHR, a web-based recruitment support system. The system helps recruiters manage resumes, extract candidate information, compare candidates with job descriptions, create shortlists, prepare outreach messages, and generate interview questions.

EasyHR follows a common web architecture with a React frontend, a FastAPI backend, and a PostgreSQL database. Several features use large language models to support resume parsing, candidate scoring, and natural language interaction with the candidate pool. The final result is a working prototype that can be deployed locally with Docker and demonstrates the main workflow of an AI-assisted recruitment platform.

Keywords: recruitment, resume management, artificial intelligence, FastAPI, React, PostgreSQL, large language model.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	3
2.1 Khảo sát hiện trạng	3
2.1.1 Quy trình tuyển dụng hiện tại	3
2.1.2 Những khó khăn trong thực tế	3
2.2 Tổng quan chức năng	3
2.2.1 Nhóm người dùng chính	3
2.2.2 Các nhóm chức năng chính	4
2.2.3 Tổng hợp chức năng theo luồng sử dụng.....	4
2.3 Đặc tả chức năng	5
2.3.1 Quản lý CV và hồ sơ ứng viên.....	5
2.3.2 Quản lý mô tả công việc	5
2.3.3 Chấm điểm ứng viên	5
2.3.4 Hỏi đáp, shortlist và các bước sau sàng lọc	5
2.4 Yêu cầu phi chức năng	5
2.4.1 Yêu cầu về bảo mật và phân quyền.....	5
2.4.2 Yêu cầu về hiệu năng và xử lý trạng thái.....	5
2.4.3 Yêu cầu về giao diện sử dụng	5
2.4.4 Yêu cầu về khả năng mở rộng.....	6
2.5 Bảng tóm tắt ca sử dụng	6

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	7
3.1 Kiến trúc ứng dụng web	7
3.2 Công nghệ sử dụng cho frontend	7
3.2.1 React	7
3.2.2 Vite	7
3.3 Công nghệ sử dụng cho backend	7
3.3.1 FastAPI và Python.....	7
3.3.2 Tổ chức backend theo lớp.....	8
3.4 Công nghệ sử dụng cho dữ liệu và triển khai	8
3.4.1 PostgreSQL	8
3.4.2 Redis và Celery.....	8
3.4.3 Docker	8
3.5 Mô hình ngôn ngữ lớn trong hệ thống	8
3.5.1 Vai trò của LLM trong EasyHR.....	8
3.5.2 Giới hạn và hướng kiểm soát	9
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	10
4.1 Thiết kế kiến trúc.....	10
4.1.1 Lựa chọn mô hình kiến trúc	10
4.1.2 Kiến trúc tổng thể	10
4.1.3 Phân chia thành phần chính.....	11
4.2 Thiết kế chi tiết.....	11
4.2.1 Mô hình dữ liệu	11
4.2.2 Thiết kế backend.....	12
4.2.3 Thiết kế frontend.....	13
4.3 Xây dựng hệ thống.....	13
4.3.1 Luồng xử lý CV	13

4.3.2	Luồng chấm điểm ứng viên.....	14
4.3.3	Các chức năng hỗ trợ.....	15
4.4	Kiểm thử.....	15
4.5	Triển khai	15
	CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	16
5.1	Tổ chức dữ liệu tuyển dụng theo luồng nghiệp vụ	16
5.1.1	Vấn đề đặt ra.....	16
5.1.2	Giải pháp được áp dụng	16
5.1.3	Đóng góp đạt được	16
5.2	Chấm điểm ứng viên có giải thích	16
5.2.1	Vấn đề đặt ra.....	16
5.2.2	Giải pháp được áp dụng	16
5.2.3	Đóng góp đạt được	17
5.3	Hỏi đáp trên tập ứng viên	17
5.3.1	Vấn đề đặt ra.....	17
5.3.2	Giải pháp được áp dụng	17
5.3.3	Đóng góp đạt được	17
5.4	Liên kết sau sàng lọc: shortlist, outreach và phỏng vấn.....	17
5.4.1	Vấn đề đặt ra.....	17
5.4.2	Giải pháp được áp dụng	17
5.4.3	Đóng góp đạt được	18
5.5	Khả năng triển khai và mở rộng	18
5.5.1	Ý nghĩa thực tế	18
5.5.2	Hướng mở rộng về sau.....	18

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	19
6.1 Kết luận.....	19
6.1.1 Kết quả đạt được	19
6.1.2 Hạn chế.....	19
6.2 Hướng phát triển.....	19
6.2.1 Về kiến trúc hệ thống.....	19
6.2.2 Về AI và dữ liệu	20
6.2.3 Về mở rộng nghiệp vụ	20
TÀI LIỆU THAM KHẢO.....	22
PHỤ LỤC.....	24
A. ĐẶC TẢ USE CASE CHÍNH	24
A.1 Use case tải lên và phân tích CV	24
A.2 Use case chấm điểm ứng viên	24
A.3 Use case tạo danh sách rút gọn.....	25
A.4 Use case chuẩn bị câu hỏi phỏng vấn	25
B. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY THỬ.....	26
B.1 Yêu cầu môi trường	26
B.2 Các bước chạy hệ thống.....	26
B.3 Địa chỉ truy cập	26
B.4 Một số lỗi thường gặp.....	26

DANH MỤC HÌNH VẼ

Hình 4.1	Kiến trúc tổng thể của hệ thống EasyHR	10
Hình 4.2	Mô hình dữ liệu khái quát của EasyHR	12
Hình 4.3	Giao diện quản lý ứng viên của EasyHR	13
Hình 4.4	Luồng xử lý CV trong hệ thống	14
Hình 4.5	Luồng chấm điểm ứng viên theo mô tả công việc	14
Hình 4.6	Kết quả chấm điểm ứng viên theo mô tả công việc	15

DANH MỤC BẢNG BIỂU

Bảng 2.1	Các ca sử dụng chính của EasyHR	6
Bảng 4.1	Một số nhóm API chính	12
Bảng A.1	Đặc tả use case tải lên CV	24
Bảng A.2	Đặc tả use case chấm điểm ứng viên	24
Bảng A.3	Đặc tả use case tạo shortlist	25
Bảng A.4	Đặc tả use case chuẩn bị phỏng vấn	25

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CV	Hồ sơ ứng tuyển (Curriculum Vitae)
JD	Mô tả công việc (Job Description)
LLM	Mô hình ngôn ngữ lớn (Large Language Model)
ORM	Kỹ thuật ánh xạ dữ liệu quan hệ sang đối tượng (Object Relational Mapping)
RBAC	Phân quyền dựa trên vai trò (Role-Based Access Control)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong nhiều doanh nghiệp, tuyển dụng không chỉ là việc đăng tin và chờ ứng viên gửi hồ sơ. Một vị trí có thể nhận hàng chục đến hàng trăm CV, trong khi nhà tuyển dụng vẫn phải đọc từng hồ sơ, ghi chú kỹ năng, đối chiếu với mô tả công việc, trao đổi với ứng viên và chuẩn bị nội dung phỏng vấn. Nếu làm hoàn toàn thủ công, quá trình này dễ mất nhiều thời gian và phụ thuộc khá nhiều vào kinh nghiệm của từng người phụ trách.

Vấn đề càng rõ hơn khi dữ liệu tuyển dụng bị phân tán. CV thường nằm trong email, thư mục máy tính hoặc các bảng tính riêng. Mô tả công việc được lưu ở một nơi khác. Kết quả đánh giá ứng viên lại có thể nằm trong ghi chú cá nhân của nhà tuyển dụng. Khi cần xem lại lý do vì sao một ứng viên được chọn hoặc bị loại, nhóm tuyển dụng thường không có đủ dữ liệu đã được chuẩn hóa để đối chiếu.

Trong bối cảnh đó, các công cụ có ứng dụng trí tuệ nhân tạo có thể hỗ trợ tốt hơn cho nhà tuyển dụng. AI không thay thế quyết định cuối cùng của con người, nhưng có thể giúp đọc nhanh CV, trích xuất thông tin chính, gợi ý mức độ phù hợp giữa ứng viên và vị trí, cũng như hỗ trợ soạn nội dung trao đổi. Điều quan trọng là hệ thống cần trình bày kết quả theo cách minh bạch, có lý do đánh giá và vấn đề người dùng kiểm soát các quyết định quan trọng.

Từ nhu cầu trên, em lựa chọn đề tài **Hệ thống hỗ trợ tuyển dụng toàn diện EasyHR**. Đề tài tập trung xây dựng một hệ thống web giúp nhà tuyển dụng quản lý toàn bộ luồng làm việc cơ bản: tiếp nhận CV, quản lý hồ sơ ứng viên, tạo mô tả công việc, chấm điểm ứng viên, tạo danh sách rút gọn, trò chuyện với dữ liệu ứng viên, chuẩn bị email và câu hỏi phỏng vấn.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu chính của đề án là xây dựng một nguyên mẫu hệ thống có thể sử dụng được trong môi trường thử nghiệm, thể hiện đầy đủ các nghiệp vụ quan trọng của một nền tảng hỗ trợ tuyển dụng. Hệ thống cần có giao diện trực quan cho nhà tuyển dụng, backend có cấu trúc rõ ràng và cơ sở dữ liệu đủ để lưu trữ các thực thể chính trong quá trình tuyển dụng.

Các mục tiêu cụ thể gồm: xây dựng chức năng tải lên và quản lý CV dạng PDF; trích xuất thông tin ứng viên thành hồ sơ có cấu trúc; quản lý mô tả công việc; chấm điểm ứng viên theo từng tiêu chí; lưu và quản lý danh sách rút gọn; hỗ trợ hỏi đáp trên tập ứng viên; quản lý nội dung liên hệ ứng viên; và tạo bộ câu hỏi phỏng

vấn theo từng cặp ứng viên - công việc.

Phạm vi đề án tập trung vào hệ thống nội bộ cho nhà tuyển dụng. Đề án chưa hướng tới việc thay thế hoàn toàn một hệ thống quản trị nhân sự lớn. Các chức năng như ký hợp đồng, tính lương, quản lý nhân sự sau tuyển dụng và tích hợp sâu với các công việc làm chưa nằm trong phạm vi chính. Ngoài ra, các kết quả do AI sinh ra được xem là gợi ý hỗ trợ, không phải quyết định tự động cuối cùng.

1.3 Định hướng giải pháp

EasyHR được phát triển theo mô hình ứng dụng web nhiều lớp. Giao diện người dùng được xây dựng bằng React để tạo các màn hình thao tác cho nhà tuyển dụng. Backend sử dụng FastAPI, một framework Python phù hợp để xây dựng API nhanh và rõ ràng [1]. Dữ liệu được lưu trong PostgreSQL [2], còn Redis và Celery hỗ trợ các tác vụ nền khi cần mở rộng xử lý bất đồng bộ.

Đối với các chức năng có yếu tố ngôn ngữ tự nhiên, hệ thống sử dụng mô hình ngôn ngữ lớn. Trong báo cáo này, thuật ngữ LLM được hiểu là mô hình AI có khả năng đọc, phân tích và sinh văn bản. LLM được dùng để phân tích nội dung CV, chấm điểm ứng viên theo mô tả công việc và hỗ trợ sinh nội dung gợi ý. Để tránh việc hệ thống trở thành một “hộp đen”, kết quả chấm điểm luôn đi kèm lý do và điểm theo từng thành phần.

Đóng góp chính của đề án là thiết kế và hiện thực một luồng tuyển dụng tương đối trọn vẹn trong một hệ thống duy nhất. Thay vì chỉ tập trung vào một chức năng riêng lẻ như đọc CV hoặc chấm điểm, EasyHR kết nối nhiều bước của quy trình tuyển dụng vào cùng một mô hình dữ liệu và giao diện làm việc.

1.4 Bố cục đề án

Phần còn lại của báo cáo được tổ chức thành các chương như sau. Chương 2 trình bày khảo sát bài toán tuyển dụng, các nhóm người dùng, yêu cầu chức năng và yêu cầu phi chức năng của hệ thống. Chương 3 giới thiệu các nền tảng lý thuyết và công nghệ được sử dụng trong quá trình xây dựng EasyHR, bao gồm kiến trúc web, cơ sở dữ liệu, mô hình ngôn ngữ lớn và các công nghệ backend, frontend.

Chương 4 trình bày quá trình phân tích thiết kế và triển khai hệ thống, từ kiến trúc tổng thể, mô hình dữ liệu, thiết kế API đến các màn hình chính. Chương 5 tập trung vào các giải pháp và đóng góp nổi bật của đề án, đặc biệt là luồng chấm điểm ứng viên, hỏi đáp trên tập ứng viên và tổ chức dữ liệu tuyển dụng. Chương 6 tổng kết kết quả đạt được, chỉ ra các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương trước đã trình bày lý do lựa chọn đề tài và mục tiêu tổng quát của EasyHR. Chương này đi sâu hơn vào bối cảnh sử dụng, các nhóm người dùng và yêu cầu của hệ thống. Việc khảo sát yêu cầu giúp xác định rõ hệ thống cần làm gì, chưa cần làm gì và các ràng buộc cần lưu ý khi triển khai.

2.1 Khảo sát hiện trạng

2.1.1 Quy trình tuyển dụng hiện tại

Một quy trình tuyển dụng phổ biến thường bắt đầu từ việc tạo mô tả công việc, đăng tin, nhận CV, lọc hồ sơ, phỏng vấn và ra quyết định. Trong phạm vi đề án, em tập trung vào giai đoạn từ khi nhà tuyển dụng đã có mô tả công việc và nhận được CV của ứng viên. Đây là giai đoạn có nhiều công việc lặp lại, cần đọc nhiều văn bản và dễ phát sinh sai sót khi số lượng hồ sơ tăng.

Qua quan sát các công cụ tuyển dụng hiện nay, có thể thấy nhiều hệ thống làm tốt phần lưu trữ và theo dõi trạng thái ứng viên. Tuy nhiên, các thao tác phân tích nội dung CV, so sánh với yêu cầu công việc và giải thích lý do đánh giá thường vẫn cần làm thủ công hoặc nằm rải rác ở các công cụ khác nhau. Điều này tạo ra khoảng trống cho một hệ thống hỗ trợ nhà tuyển dụng bằng AI nhưng vẫn giữ được dữ liệu có cấu trúc.

2.1.2 Những khó khăn trong thực tế

Từ quy trình trên, có thể thấy một số khó khăn thường gặp. Thứ nhất, dữ liệu tuyển dụng thường bị phân tán giữa email, thư mục máy tính, bảng tính và các hệ thống nhỏ lẻ khác nhau. Thứ hai, nhà tuyển dụng phải đọc nhiều CV trong thời gian ngắn nên dễ bỏ sót thông tin quan trọng. Thứ ba, việc đánh giá mức độ phù hợp giữa CV và mô tả công việc thường phụ thuộc nhiều vào kinh nghiệm cá nhân, dẫn đến khó giải thích lại khi cần xem xét.

Đối với các doanh nghiệp nhỏ và vừa, những khó khăn này càng rõ hơn vì nguồn lực tuyển dụng có hạn. Do đó, một hệ thống hỗ trợ theo hướng gom dữ liệu, chuẩn hóa hồ sơ và đưa ra gợi ý đánh giá có thể giúp giảm tải đáng kể cho người dùng.

2.2 Tổng quan chức năng

2.2.1 Nhóm người dùng chính

Người dùng chính của EasyHR là nhà tuyển dụng hoặc nhân sự phụ trách sàng lọc hồ sơ. Nhóm người dùng này cần thao tác nhanh với nhiều CV, xem được thông tin ứng viên ở dạng dễ đọc và có cơ sở để chọn ra nhóm ứng viên phù hợp. Ngoài ra, trưởng nhóm tuyển dụng hoặc quản lý nhân sự cũng có thể dùng hệ thống để

xem danh sách rút gọn, theo dõi kết quả chấm điểm và kiểm tra quá trình liên hệ ứng viên.

Trong thiết kế hiện tại, hệ thống có khái niệm tài khoản và vai trò như admin, recruiter và viewer. Admin có thể quản lý tài khoản và cấu hình hệ thống. Recruiter là người thực hiện phần lớn nghiệp vụ tuyển dụng. Viewer chỉ xem dữ liệu và không thực hiện các thao tác thay đổi quan trọng. Cách chia vai trò này giúp hệ thống có nền tảng để mở rộng phân quyền trong tương lai.

2.2.2 Các nhóm chức năng chính

Từ góc nhìn người dùng, EasyHR được tổ chức theo các nhóm chức năng chính gồm tiếp nhận CV, quản lý hồ sơ ứng viên, quản lý mô tả công việc, chấm điểm ứng viên, hỏi đáp trên tập ứng viên, tạo danh sách rút gọn và chuẩn bị các bước tiếp theo sau sàng lọc. Các nhóm chức năng này tạo thành một luồng làm việc tương đối liền mạch thay vì tách rời thành nhiều công cụ nhỏ.

Bảng 2.1 tóm tắt một số ca sử dụng (use case, tức là tình huống sử dụng điển hình) của hệ thống.

2.2.3 Tổng hợp chức năng theo luồng sử dụng

Từ phạm vi đề tài, hệ thống cần đáp ứng các nhóm chức năng chính sau:

- Quản lý CV: tải lên nhiều file PDF, kiểm tra định dạng, lưu trạng thái xử lý và xem danh sách CV đã nhập.
- Quản lý hồ sơ ứng viên: lưu thông tin được trích xuất từ CV như họ tên, email, số điện thoại, học vấn, kinh nghiệm, kỹ năng, dự án và chứng chỉ.
- Quản lý mô tả công việc: tạo, sửa, xem danh sách và đánh dấu trạng thái hoạt động của từng mô tả công việc.
- Chấm điểm ứng viên: so khớp một hoặc nhiều ứng viên với một mô tả công việc, cho phép cấu hình trọng số tiêu chí và ngưỡng đạt.
- Hỏi đáp với tập ứng viên: cho phép nhà tuyển dụng đặt câu hỏi bằng ngôn ngữ tự nhiên để lọc hoặc tìm ứng viên phù hợp.
- Quản lý danh sách rút gọn: tạo bộ sưu tập ứng viên từ kết quả tìm kiếm hoặc kết quả chấm điểm.
- Hỗ trợ liên hệ và phỏng vấn: lưu nội dung email, trạng thái gửi và bộ câu hỏi phỏng vấn theo từng ứng viên.

2.3 Đặc tả chức năng

2.3.1 Quản lý CV và hồ sơ ứng viên

Hệ thống cần cho phép nhà tuyển dụng tải lên một hoặc nhiều CV dưới dạng PDF. Sau khi tải lên, hệ thống phải lưu được thông tin tài liệu, trạng thái xử lý và nội dung hồ sơ ứng viên đã được trích xuất. Người dùng cần xem lại danh sách CV, tra cứu chi tiết từng hồ sơ và xóa những dữ liệu không còn cần thiết.

2.3.2 Quản lý mô tả công việc

Nhà tuyển dụng cần tạo, sửa, xem và quản lý trạng thái của từng mô tả công việc. Đây là dữ liệu đầu vào quan trọng cho bước chấm điểm, nên hệ thống cần trình bày phần này rõ ràng và cho phép cập nhật khi yêu cầu tuyển dụng thay đổi.

2.3.3 Chấm điểm ứng viên

Hệ thống cần hỗ trợ so khớp một hoặc nhiều ứng viên với một mô tả công việc. Kết quả không chỉ là điểm tổng mà còn nên có giải thích ngắn và điểm theo từng tiêu chí chính. Ngoài ra, người dùng cần có khả năng điều chỉnh trọng số hoặc ngưỡng đánh giá để phù hợp với từng vị trí tuyển dụng.

2.3.4 Hỏi đáp, shortlist và các bước sau sàng lọc

Sau khi đã có dữ liệu ứng viên, hệ thống cần hỗ trợ hỏi đáp bằng ngôn ngữ tự nhiên để tìm ứng viên phù hợp nhanh hơn. Từ kết quả đó, nhà tuyển dụng có thể tạo danh sách rút gọn, lưu nội dung liên hệ và chuẩn bị bộ câu hỏi phỏng vấn. Đây là phần giúp EasyHR khác với một hệ thống chỉ dừng ở bước lưu CV.

2.4 Yêu cầu phi chức năng

2.4.1 Yêu cầu về bảo mật và phân quyền

Bên cạnh chức năng, EasyHR cần đáp ứng một số yêu cầu phi chức năng cơ bản. Trước hết, dữ liệu ứng viên là dữ liệu nhạy cảm nên hệ thống cần định hướng thiết kế có phân quyền, kiểm soát truy cập và thời hạn lưu trữ hợp lý. Các thao tác như xem hồ sơ, sửa dữ liệu hoặc xóa tài liệu cần được giới hạn theo vai trò người dùng.

2.4.2 Yêu cầu về hiệu năng và xử lý trạng thái

Các API cần có kiểm tra đầu vào rõ ràng, đặc biệt với file tải lên, mã định danh và dữ liệu do người dùng nhập. Với các tác vụ có thể mất nhiều thời gian như phân tích CV và chấm điểm, hệ thống cần trả về trạng thái xử lý rõ ràng để người dùng biết tiến trình đang diễn ra đến đâu.

2.4.3 Yêu cầu về giao diện sử dụng

Về trải nghiệm sử dụng, giao diện cần dễ theo dõi và giảm số bước thao tác không cần thiết. Người dùng cần biết hệ thống đang làm gì, có lỗi ở file nào và kết

quả cuối cùng ra sao. Những màn hình quan trọng như danh sách ứng viên, kết quả chấm điểm và shortlist cần được trình bày dễ đọc để thuận tiện cho việc so sánh.

2.4.4 Yêu cầu về khả năng mở rộng

Về triển khai và phát triển lâu dài, hệ thống cần có cấu trúc đủ để mở rộng vì quy trình tuyển dụng có thể bổ sung nhiều bước mới. Việc đóng gói bằng Docker cũng là một yêu cầu thực tế để giảm khác biệt môi trường giữa các máy phát triển và máy chạy thử.

2.5 Bảng tóm tắt ca sử dụng

Bảng 2.1 tóm tắt một số ca sử dụng chính của hệ thống.

Bảng 2.1: Các ca sử dụng chính của EasyHR

Ca sử dụng	Mô tả ngắn
Tải lên CV	Nhà tuyển dụng chọn một hoặc nhiều file PDF, hệ thống lưu file và trích xuất thông tin ứng viên.
Tạo mô tả công việc	Nhà tuyển dụng nhập nội dung yêu cầu tuyển dụng để dùng cho bước chấm điểm.
Chấm điểm ứng viên	Hệ thống so sánh hồ sơ ứng viên với mô tả công việc và trả về điểm cùng lý do đánh giá.
Tạo shortlist	Nhà tuyển dụng lưu nhóm ứng viên tiềm năng vào một danh sách rút gọn để theo dõi tiếp.
Chuẩn bị phỏng vấn	Hệ thống lưu hoặc gợi ý bộ câu hỏi phỏng vấn dựa trên ứng viên và vị trí tuyển dụng.

Từ các yêu cầu trên, chương tiếp theo sẽ trình bày các công nghệ được lựa chọn để xây dựng hệ thống và lý do lựa chọn từng nhóm công nghệ.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Sau khi xác định yêu cầu hệ thống, chương này trình bày các nền tảng lý thuyết và công nghệ được dùng để hiện thực EasyHR. Việc lựa chọn công nghệ được thực hiện theo hướng ưu tiên tính rõ ràng, dễ triển khai và phù hợp với một hệ thống web có sử dụng AI ở một số nghiệp vụ.

3.1 Kiến trúc ứng dụng web

EasyHR được xây dựng theo kiến trúc client - server. Phần client là giao diện chạy trên trình duyệt, chịu trách nhiệm hiển thị dữ liệu và nhận thao tác của người dùng. Phần server cung cấp API để xử lý nghiệp vụ, truy xuất cơ sở dữ liệu và gọi các dịch vụ AI khi cần. Cách tổ chức này giúp tách biệt phần giao diện với phần xử lý dữ liệu, từ đó dễ bảo trì và kiểm thử hơn.

Trong hệ thống, frontend giao tiếp với backend thông qua các API dạng REST. REST là phong cách thiết kế API sử dụng các phương thức HTTP như GET, POST, PATCH và DELETE để thao tác với tài nguyên. Ví dụ, CV, mô tả công việc, danh sách rút gọn và bộ câu hỏi phỏng vấn đều được xem như các tài nguyên riêng.

3.2 Công nghệ sử dụng cho frontend

3.2.1 React

Phần giao diện sử dụng React, một thư viện phổ biến để xây dựng giao diện người dùng theo hướng component [3]. Mỗi màn hình của EasyHR được tách thành các thành phần nhỏ như bảng dữ liệu, hộp thoại tải CV, thẻ trạng thái, biểu đồ điểm và form nhập mô tả công việc. Cách tách này giúp tái sử dụng giao diện và giảm lặp mã.

3.2.2 Vite

Vite được dùng làm công cụ phát triển frontend vì thời gian khởi động nhanh và hỗ trợ tốt cho ứng dụng React [4]. Trong quá trình phát triển, Vite giúp việc kiểm tra thay đổi giao diện diễn ra nhanh hơn so với các công cụ đóng gói truyền thống.

3.3 Công nghệ sử dụng cho backend

3.3.1 FastAPI và Python

Backend của EasyHR sử dụng FastAPI. Đây là framework Python hiện đại, hỗ trợ khai báo kiểu dữ liệu rõ ràng và tự động sinh tài liệu API tương tác [1]. Điểm phù hợp của FastAPI với đề án là cú pháp gọn, dễ chia module theo từng nhóm endpoint và thuận tiện khi kết hợp với Pydantic để kiểm tra dữ liệu đầu vào.

Python được lựa chọn vì có hệ sinh thái tốt cho xử lý văn bản, gọi mô hình AI và

xây dựng dịch vụ web. Trong EasyHR, Python được dùng cho các lớp API, service xử lý nghiệp vụ, mô hình dữ liệu và các tác vụ nền.

3.3.2 Tổ chức backend theo lớp

Về mặt tổ chức, backend được chia thành các lớp chính như endpoint, service, schema và model. Endpoint nhận yêu cầu từ phía người dùng, kiểm tra dữ liệu đầu vào và trả kết quả. Service xử lý phần nghiệp vụ chính. Schema mô tả dữ liệu trao đổi giữa các phần của hệ thống. Model biểu diễn dữ liệu lưu trong cơ sở dữ liệu. Cách chia này giúp mã nguồn dễ đọc và dễ kiểm thử hơn.

3.4 Công nghệ sử dụng cho dữ liệu và triển khai

3.4.1 PostgreSQL

PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính [2]. Dữ liệu tuyển dụng có nhiều quan hệ rõ ràng: một CV tương ứng với một hồ sơ ứng viên, một lần chấm điểm gắn với một mô tả công việc, một danh sách rút gọn chứa nhiều ứng viên. Vì vậy, cơ sở dữ liệu quan hệ là lựa chọn phù hợp để bảo đảm tính nhất quán và truy vấn dữ liệu.

Các bảng chính của hệ thống gồm tài khoản người dùng, tài liệu CV, hồ sơ ứng viên, mô tả công việc, lần chấm điểm, kết quả chấm điểm, phiên hỏi đáp, danh sách rút gọn, nội dung liên hệ và bộ câu hỏi phỏng vấn. Hệ thống dùng migration (quản lý thay đổi cấu trúc cơ sở dữ liệu theo phiên bản) để cập nhật schema trong quá trình phát triển.

3.4.2 Redis và Celery

Redis được dùng làm hạ tầng hỗ trợ cho hàng đợi và lưu trữ tạm thời. Celery là công cụ xử lý tác vụ nền trong Python [5]. Trong phiên bản hiện tại, một số luồng xử lý vẫn còn đồng bộ, nhưng việc chuẩn bị Celery giúp hệ thống có thể chuyển các tác vụ nặng như phân tích nhiều CV hoặc chấm điểm hàng loạt sang xử lý nền trong các phiên bản sau.

3.4.3 Docker

Docker được dùng để đóng gói môi trường chạy của hệ thống [6]. Với Docker Compose, các thành phần như PostgreSQL, Redis, backend, worker và frontend có thể được khởi động cùng nhau. Điều này giảm lỗi do khác biệt môi trường cài đặt giữa các máy.

3.5 Mô hình ngôn ngữ lớn trong hệ thống

3.5.1 Vai trò của LLM trong EasyHR

EasyHR sử dụng LLM ở các nghiệp vụ cần hiểu hoặc sinh văn bản. Khi phân tích CV, LLM hỗ trợ chuyển nội dung văn bản tự do thành các trường có cấu trúc

như kỹ năng, kinh nghiệm và học vấn. Khi chấm điểm, LLM đọc mô tả công việc và hồ sơ ứng viên để đưa ra điểm theo tiêu chí. Khi chuẩn bị liên hệ hoặc phỏng vấn, LLM có thể sinh nội dung gợi ý để nhà tuyển dụng chỉnh sửa.

3.5.2 Giới hạn và hướng kiểm soát

Tuy nhiên, LLM có thể sinh kết quả chưa chính xác hoặc thiếu nhất quán. Vì vậy hệ thống không chỉ lưu điểm tổng, mà còn lưu điểm theo từng thành phần và phần giải thích. Cách này giúp nhà tuyển dụng kiểm tra lại kết quả thay vì phụ thuộc hoàn toàn vào AI.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày phần thiết kế và triển khai hệ thống EasyHR. Nội dung được mô tả theo thứ tự từ kiến trúc tổng thể, mô hình dữ liệu, các module backend đến giao diện người dùng. Do báo cáo chưa gắn ảnh chụp màn hình cuối cùng, các hình trong chương này được đặt dưới dạng placeholder để có thể thay thế sau.

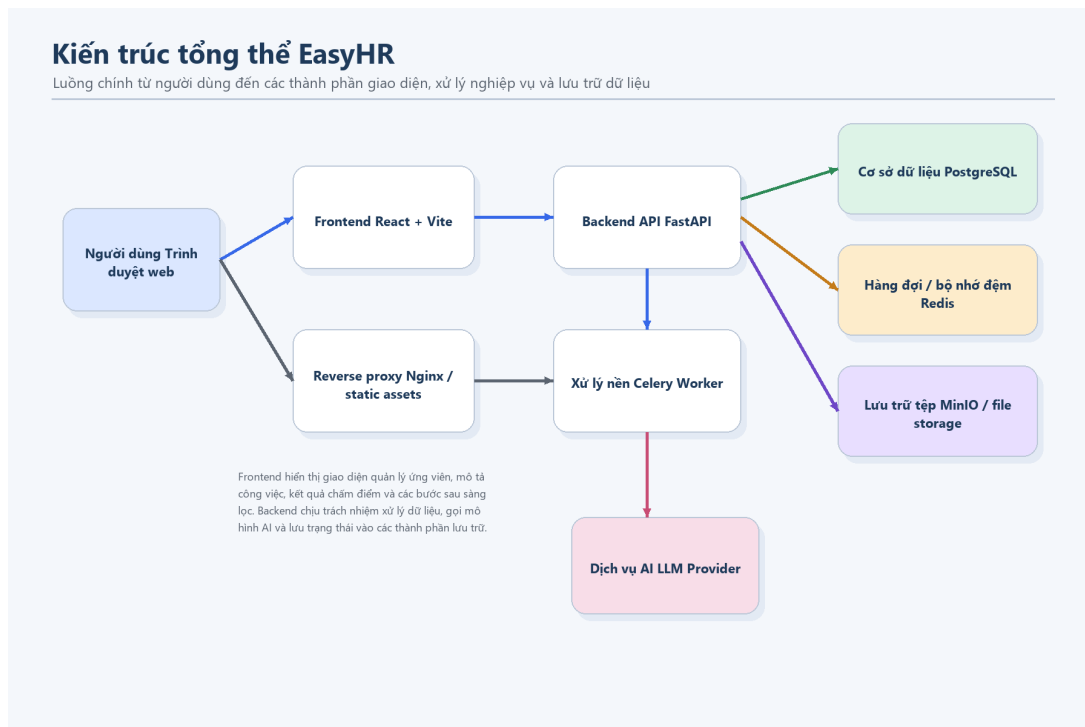
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn mô hình kiến trúc

EasyHR được thiết kế theo mô hình client - server. Cách tổ chức này phù hợp với một hệ thống web có nhiều màn hình quản trị và có backend chịu trách nhiệm xử lý dữ liệu. Mô hình này cũng giúp tách rõ phần giao diện, phần xử lý nghiệp vụ và phần lưu trữ dữ liệu, từ đó thuận tiện cho bảo trì và mở rộng.

4.1.2 Kiến trúc tổng thể

EasyHR gồm năm thành phần chính: frontend, backend, cơ sở dữ liệu PostgreSQL, Redis và worker xử lý nền. Người dùng thao tác trên trình duyệt. Frontend gửi yêu cầu tới backend thông qua API. Backend kiểm tra dữ liệu, xử lý nghiệp vụ, đọc ghi PostgreSQL và gọi LLM khi cần. Redis và worker được chuẩn bị cho các tác vụ có thời gian xử lý dài.



Hình 4.1: Kiến trúc tổng thể của hệ thống EasyHR

Kiến trúc này có ưu điểm là tách trách nhiệm rõ ràng. Frontend không trực tiếp

truy cập cơ sở dữ liệu. Backend đóng vai trò trung tâm kiểm soát nghiệp vụ. Các tác vụ nặng có thể được chuyển sang worker trong tương lai mà không làm thay đổi nhiều giao diện người dùng.

4.1.3 Phân chia thành phần chính

Ở mức tổng quan, frontend chịu trách nhiệm hiển thị dữ liệu và nhận thao tác của người dùng. Backend tiếp nhận yêu cầu, gọi các service cần thiết và trả kết quả về giao diện. PostgreSQL lưu dữ liệu chính, còn Redis hỗ trợ các tác vụ tạm thời và hàng đợi. Worker là thành phần có thể nhận những việc nặng như phân tích hàng loạt hoặc chấm điểm nhiều hồ sơ.

4.2 Thiết kế chi tiết

4.2.1 Mô hình dữ liệu

Mô hình dữ liệu của EasyHR được thiết kế xoay quanh hồ sơ ứng viên và các hoạt động tuyển dụng liên quan. Bảng ResumeDocument lưu thông tin file CV đã tải lên, trạng thái xử lý và thời hạn lưu trữ. Bảng CandidateProfile lưu dữ liệu có cấu trúc được trích xuất từ CV. Bảng JobDescription lưu mô tả công việc. Khi thực hiện chấm điểm, hệ thống tạo MatchRun và nhiều MatchResult tương ứng với từng ứng viên.

Bên cạnh đó, hệ thống có các bảng phục vụ luồng làm việc của nhà tuyển dụng như QuerySession, QueryTurn, ShortlistCollection, ShortlistItem, OutreachMessage và InterviewQuestionSet. Cách thiết kế này giúp các bước sau khi lọc ứng viên không bị tách rời khỏi dữ liệu gốc.



Hình 4.2: Mô hình dữ liệu khái quát của EasyHR

4.2.2 Thiết kế backend

Backend được chia theo các nhóm endpoint chính. Nhóm upload xử lý tải CV và quản lý tài liệu đã tải lên. Nhóm job descriptions quản lý mô tả công việc. Nhóm score thực hiện chấm điểm ứng viên. Nhóm chat hỗ trợ hỏi đáp trên tập ứng viên. Các nhóm shortlist, outreach và interview questions hỗ trợ những bước sau sàng lọc.

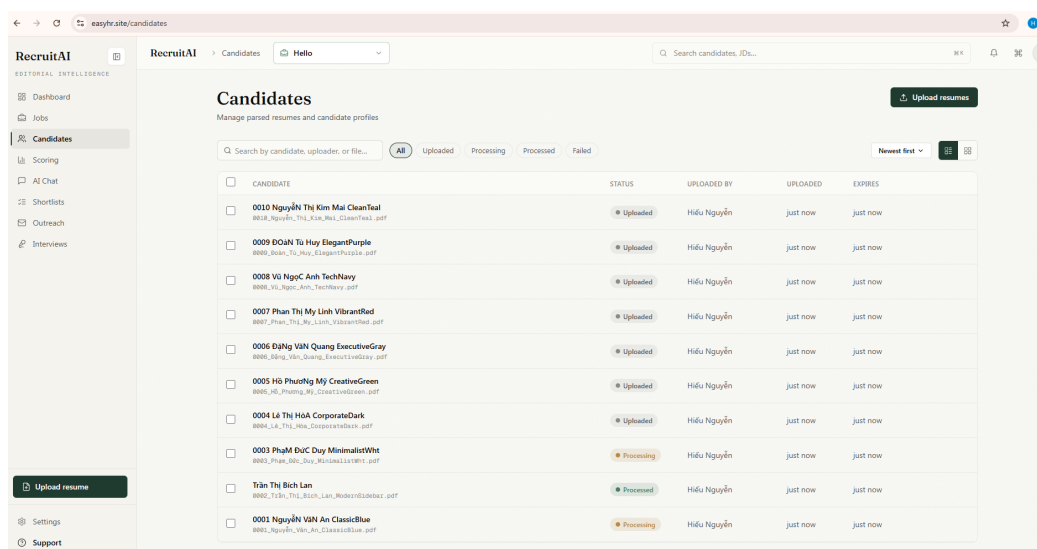
Ở tầng service, mỗi nhóm nghiệp vụ được tách thành module riêng. Ví dụ, resume service xử lý logic liên quan đến CV và hồ sơ ứng viên; public job service xử lý luồng ứng tuyển công khai; score candidate service xử lý đánh giá ứng viên; interview service xử lý bộ câu hỏi và phiên phỏng vấn. Cách tổ chức này giúp endpoint chỉ đóng vai trò nhận yêu cầu và trả phản hồi, còn logic chính nằm trong service.

Bảng 4.1: Một số nhóm API chính

Nhóm API	Chức năng
/api/v1/upload	Tải lên, xem, sửa và xóa CV.
/api/v1/job-descriptions	Quản lý mô tả công việc.
/api/v1/score	Chấm điểm ứng viên theo mô tả công việc.
/api/v1/chat	Hỏi đáp với dữ liệu ứng viên.
/api/v1/shortlist	Quản lý phiên tìm kiếm và danh sách rút gọn.
/api/v1/outreach	Quản lý nội dung liên hệ ứng viên.

4.2.3 Thiết kế frontend

Giao diện frontend được thiết kế theo luồng làm việc của nhà tuyển dụng. Các màn hình chính gồm dashboard, danh sách ứng viên, chi tiết ứng viên, quản lý mô tả công việc, chấm điểm, AI chat, shortlist, outreach và phỏng vấn. Hệ thống dùng layout chung với thanh điều hướng bên trái và vùng nội dung chính để người dùng dễ chuyển giữa các bước.



Hình 4.3: Giao diện quản lý ứng viên của EasyHR

Một số màn hình được chú trọng hơn vì có ảnh hưởng trực tiếp đến trải nghiệm sử dụng. Màn hình upload CV cần thể hiện trạng thái xử lý rõ ràng. Màn hình scoring cần hiển thị điểm tổng, điểm thành phần và lý do đánh giá. Màn hình chat cần trình bày câu trả lời dễ đọc và cho biết số lượng ứng viên phù hợp với truy vấn.

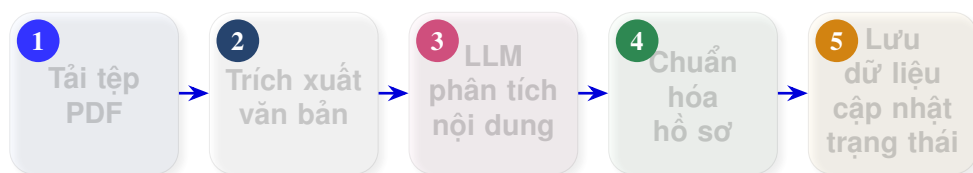
4.3 Xây dựng hệ thống

4.3.1 Luồng xử lý CV

Khi người dùng tải lên CV, hệ thống kiểm tra định dạng file PDF, lưu thông tin tài liệu và chuyển nội dung cho bước phân tích. Sau khi phân tích xong, hệ thống tạo hoặc cập nhật CandidateProfile. Các trường thông tin như kỹ năng, kinh nghiệm, học vấn và dự án được lưu riêng để phục vụ tìm kiếm và chấm điểm.

Luồng xử lý CV

Chuỗi bước chính trong quá trình tiếp nhận, phân tích và chuẩn hóa dữ liệu ứng viên



Luồng này mô tả từ lúc nhà tuyển dụng tải tệp PDF đến khi hệ thống sinh hồ sơ ứng viên có cấu trúc để phục vụ tìm kiếm và chấm điểm.

Hình 4.4: Luồng xử lý CV trong hệ thống

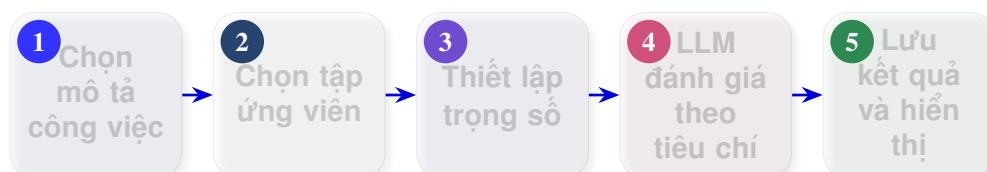
Trong phiên bản hiện tại, bước phân tích CV có thể mất thời gian khi số lượng file lớn. Vì vậy giao diện cần thông báo rõ trạng thái đang xử lý và kết quả từng file. Đây cũng là một điểm cần cải tiến sang xử lý bất đồng bộ trong tương lai.

4.3.2 Luồng chấm điểm ứng viên

Luồng chấm điểm bắt đầu khi nhà tuyển dụng chọn một mô tả công việc, chọn tập ứng viên và cấu hình trọng số. Hệ thống lấy dữ liệu ứng viên, gửi từng nhóm ứng viên cho LLM để đánh giá và lưu kết quả vào MatchResult. Mỗi kết quả gồm điểm tổng, trạng thái vượt ngưỡng, lý do đánh giá và điểm theo từng tiêu chí.

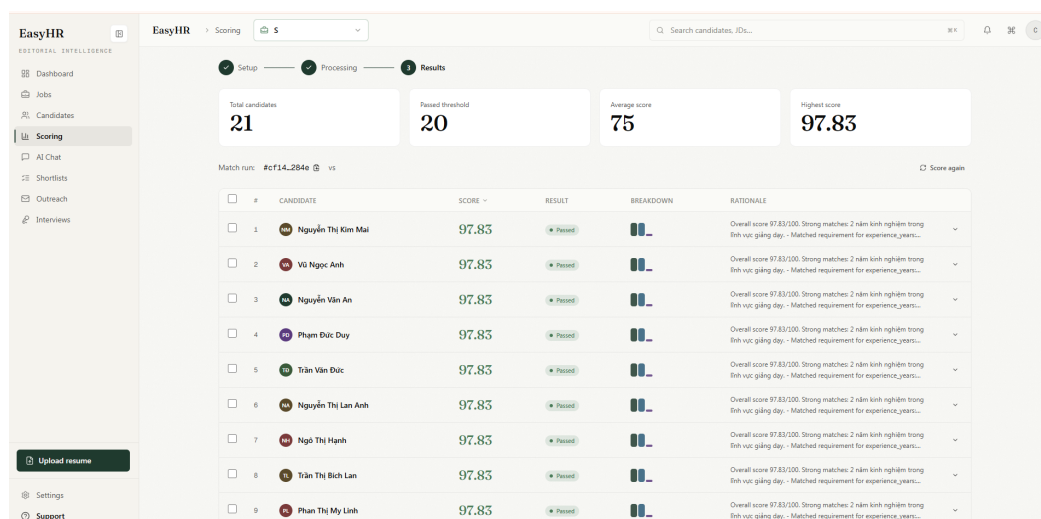
Luồng chấm điểm ứng viên

Chuỗi bước từ chọn mô tả công việc đến sinh kết quả đánh giá có giải thích



Luồng này cho thấy cách hệ thống lấy mô tả công việc và danh sách ứng viên, gửi dữ liệu cho mô hình AI, sau đó lưu lại điểm số và phần giải thích để nhà tuyển dụng xem lại.

Hình 4.5: Luồng chấm điểm ứng viên theo mô tả công việc



Hình 4.6: Kết quả chấm điểm ứng viên theo mô tả công việc

Điểm mạnh của thiết kế này là nhà tuyển dụng có thể điều chỉnh trọng số theo vị trí tuyển dụng. Ví dụ, với vị trí kỹ sư phần mềm nhiều kinh nghiệm, tiêu chí kinh nghiệm và kỹ năng có thể được đặt trọng số cao hơn. Với vị trí thực tập sinh, tiêu chí học vấn và dự án cá nhân có thể quan trọng hơn.

4.3.3 Các chức năng bổ trợ

Ngoài hai luồng chính ở trên, hệ thống còn triển khai các chức năng bổ trợ như hỏi đáp trên tập ứng viên, quản lý shortlist, chuẩn bị nội dung liên hệ và bộ câu hỏi phỏng vấn. Những chức năng này giúp kết quả sàng lọc được nối tiếp sang các bước làm việc tiếp theo, thay vì dừng lại ở một bảng điểm.

4.4 Kiểm thử

Trong quá trình phát triển, hệ thống được kiểm thử ở cả backend và frontend. Backend có các test cho endpoint xác thực, upload, job chat, chấm điểm, public job, outreach, shortlist và interview questions. Frontend có kiểm thử end-to-end cho một số luồng như ứng tuyển công khai và smoke test workspace. Các kiểm thử này giúp phát hiện lỗi hồi quy khi bổ sung chức năng mới.

4.5 Triển khai

Kết quả đạt được là hệ thống có thể khởi động bằng Docker Compose, cung cấp API qua FastAPI và giao diện frontend qua Vite. Các chức năng chính của quy trình tuyển dụng đã được kết nối với nhau ở mức nguyên mẫu hoạt động được. Dù chưa phải một sản phẩm thương mại hoàn chỉnh, EasyHR đã thể hiện được cách AI có thể hỗ trợ nhà tuyển dụng trong một luồng công việc thực tế.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày cách EasyHR được thiết kế và triển khai. Chương này tập trung hơn vào các giải pháp và đóng góp nổi bật của đề án, tức là những phần thể hiện rõ giá trị của hệ thống so với một ứng dụng quản lý hồ sơ thông thường.

5.1 Tổ chức dữ liệu tuyển dụng theo luồng nghiệp vụ

5.1.1 Vấn đề đặt ra

Trong nhiều hệ thống tuyển dụng nhỏ, dữ liệu thường chỉ dừng ở mức lưu file CV và một vài ghi chú rời rạc. Khi cần xem lại ứng viên, nhà tuyển dụng phải mở lại nhiều nguồn dữ liệu khác nhau để ghép thông tin. Điều này làm quá trình theo dõi mất thời gian và khó kiểm tra lại lý do ra quyết định.

5.1.2 Giải pháp được áp dụng

Đóng góp đầu tiên của đề án là tổ chức dữ liệu tuyển dụng theo một luồng tương đối đầy đủ. Thay vì chỉ lưu file CV, hệ thống tách riêng tài liệu CV và hồ sơ ứng viên đã được chuẩn hóa. Từ hồ sơ ứng viên, hệ thống tiếp tục liên kết tới kết quả chấm điểm, danh sách rút gọn, nội dung liên hệ và bộ câu hỏi phỏng vấn.

5.1.3 Đóng góp đạt được

Cách tổ chức này giúp dữ liệu không bị rời rạc. Khi xem một ứng viên, nhà tuyển dụng có thể lần theo các thông tin liên quan như ứng viên đã được chấm với vị trí nào, điểm mạnh được hệ thống ghi nhận là gì, đã được đưa vào shortlist nào và đã có nội dung liên hệ hay chưa. Đây là nền tảng quan trọng nếu hệ thống được mở rộng thành sản phẩm thực tế.

5.2 Chấm điểm ứng viên có giải thích

5.2.1 Vấn đề đặt ra

Nếu hệ thống chỉ trả về một điểm tổng, người dùng sẽ khó hiểu vì sao một ứng viên được xếp cao hơn ứng viên khác. Điều này làm giảm độ tin cậy của hệ thống, nhất là khi kết quả có sự tham gia của AI.

5.2.2 Giải pháp được áp dụng

Một điểm quan trọng của EasyHR là không chỉ trả về một con số điểm tổng. Hệ thống lưu điểm theo từng tiêu chí, trọng số, điểm thành phần và phần giải thích. Thiết kế này giúp nhà tuyển dụng hiểu vì sao ứng viên được đánh giá cao hoặc thấp, từ đó có thể kiểm tra lại kết quả.

5.2.3 Đóng góp đạt được

Việc cho phép cấu hình trọng số cũng làm cho hệ thống linh hoạt hơn. Mỗi vị trí tuyển dụng có mức ưu tiên khác nhau. Một vị trí backend có thể cần kinh nghiệm hệ thống và cơ sở dữ liệu, trong khi vị trí frontend cần chú ý hơn tới kỹ năng giao diện và trải nghiệm người dùng. Nếu trọng số bị cố định, kết quả chấm điểm sẽ khó phản ánh đúng nhu cầu tuyển dụng.

5.3 Hỏi đáp trên tập ứng viên

5.3.1 Vấn đề đặt ra

Khi số lượng ứng viên lớn, việc dùng bộ lọc cứng có thể không đủ linh hoạt. Nhà tuyển dụng thường muốn tìm theo cách diễn đạt tự nhiên, ví dụ như tìm ứng viên có nền tảng dữ liệu tốt hoặc từng làm dự án web thực tế.

5.3.2 Giải pháp được áp dụng

Chức năng AI chat giúp nhà tuyển dụng đặt câu hỏi trực tiếp về tập ứng viên, ví dụ tìm ứng viên có kinh nghiệm Python, đã làm dự án web hoặc có nền tảng dữ liệu. Thay vì phải kết hợp nhiều bộ lọc thủ công, người dùng có thể diễn đạt nhu cầu bằng ngôn ngữ tự nhiên. Hệ thống trả về câu trả lời và số lượng ứng viên phù hợp trong phạm vi dữ liệu hiện có.

5.3.3 Đóng góp đạt được

Trong đồ án, chức năng này được xem là một hướng tiếp cận giúp giảm khoảng cách giữa dữ liệu có cấu trúc và nhu cầu tìm kiếm thực tế của nhà tuyển dụng. Tuy nhiên, kết quả hỏi đáp vẫn cần được kiểm chứng bằng hồ sơ gốc, đặc biệt với các tiêu chí nhạy cảm hoặc có ảnh hưởng lớn tới quyết định tuyển dụng.

5.4 Liên kết sau sàng lọc: shortlist, outreach và phỏng vấn

5.4.1 Vấn đề đặt ra

Nhiều hệ thống dừng lại ở bước lọc hồ sơ, trong khi thực tế nhà tuyển dụng còn cần lưu nhóm ứng viên tiềm năng, liên hệ và chuẩn bị phỏng vấn. Nếu các bước này nằm ở công cụ khác, luồng làm việc sẽ bị đứt đoạn.

5.4.2 Giải pháp được áp dụng

Để giải quyết điểm này, EasyHR bổ sung các module shortlist, outreach và interview questions để nối tiếp luồng làm việc sau khi có kết quả chấm điểm. Cách làm này giúp dữ liệu sau sàng lọc không bị rơi ra ngoài hệ thống và người dùng có thể tiếp tục thao tác trên cùng một nguồn dữ liệu.

5.4.3 Đóng góp đạt được

Shortlist giúp lưu lại nhóm ứng viên theo từng mục đích. Outreach giúp lưu nội dung liên hệ, trạng thái gửi và nguồn tạo nội dung. Interview questions giúp chuẩn bị bộ câu hỏi theo từng ứng viên và mô tả công việc. Ba module này chưa quá phức tạp, nhưng giúp hệ thống có tính toàn diện hơn và sát với quy trình tuyển dụng thực tế.

5.5 Khả năng triển khai và mở rộng

5.5.1 Ý nghĩa thực tế

Đồ án được đóng gói bằng Docker Compose, gồm backend, worker, frontend, PostgreSQL và Redis. Cách triển khai này giúp người khác có thể chạy thử hệ thống mà không cần cài đặt thủ công quá nhiều thành phần. Ngoài ra, việc tách backend thành các endpoint và service riêng giúp hệ thống có thể mở rộng từng module mà không phải viết lại toàn bộ.

5.5.2 Hướng mở rộng về sau

Một ví dụ là luồng phân tích CV và chấm điểm hiện vẫn có phần xử lý đồng bộ. Khi cần mở rộng, các tác vụ này có thể được đưa sang Celery worker, trong khi API chỉ trả về mã tác vụ và trạng thái xử lý. Giao diện frontend khi đó chỉ cần theo dõi tiến độ thay vì chờ một request dài.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

6.1.1 Kết quả đạt được

Đồ án đã xây dựng được nguyên mẫu hệ thống EasyHR phục vụ quy trình tuyển dụng với các chức năng chính: tải lên và quản lý CV, trích xuất hồ sơ ứng viên, quản lý mô tả công việc, chấm điểm ứng viên theo tiêu chí, hỏi đáp với tập ứng viên, tạo danh sách rút gọn, quản lý nội dung liên hệ và chuẩn bị câu hỏi phỏng vấn. Hệ thống có kiến trúc web rõ ràng, sử dụng React ở frontend, FastAPI ở backend và PostgreSQL cho lưu trữ dữ liệu.

Về mặt kỹ thuật, đồ án đã thiết kế được mô hình dữ liệu tương đối đầy đủ cho nghiệp vụ tuyển dụng. Các API được chia theo từng nhóm chức năng, thuận tiện cho frontend sử dụng và kiểm thử. Hệ thống cũng đã chuẩn bị hạ tầng Docker, Redis và Celery để có thể mở rộng xử lý nền trong tương lai.

Về mặt ứng dụng, EasyHR cho thấy cách AI có thể hỗ trợ nhà tuyển dụng trong các công việc đọc hiểu văn bản và đưa ra gợi ý. Điểm quan trọng là hệ thống không chỉ sinh kết quả, mà còn lưu lại lý do đánh giá và dữ liệu liên quan để người dùng kiểm tra.

6.1.2 Hạn chế

Do thời gian thực hiện có hạn, hệ thống vẫn còn một số hạn chế. Thứ nhất, một số tác vụ như phân tích CV và chấm điểm ứng viên còn xử lý đồng bộ, có thể gây chờ lâu khi số lượng hồ sơ lớn. Thứ hai, cơ chế xác thực và phân quyền mới ở mức nền tảng, chưa được áp dụng chặt chẽ cho toàn bộ endpoint. Thứ ba, chất lượng kết quả từ LLM chưa được đánh giá bằng một bộ dữ liệu chuẩn có nhãn, nên chưa thể kết luận định lượng về độ chính xác.

Ngoài ra, phần giao diện vẫn có thể cải thiện thêm về trải nghiệm xử lý lỗi, biểu đồ trực quan và khả năng xem lại lịch sử thao tác. Một số module như outreach và interview questions mới dừng ở mức quản lý nội dung, chưa tích hợp trực tiếp với hệ thống gửi email hoặc lịch phỏng vấn.

6.2 Hướng phát triển

6.2.1 Về kiến trúc hệ thống

Trong tương lai, EasyHR có thể được phát triển theo một số hướng. Trước hết, các tác vụ nặng nên được chuyển hoàn toàn sang xử lý bất đồng bộ bằng worker, kèm theo cơ chế theo dõi tiến độ theo thời gian thực. Tiếp theo, hệ thống cần hoàn thiện xác thực, phân quyền, mã hóa dữ liệu nhạy cảm và nhật ký truy cập để phù

hợp hơn với dữ liệu tuyển dụng thực tế.

6.2.2 Về AI và dữ liệu

Về AI, hệ thống cần có cơ chế đánh giá chất lượng kết quả bằng tập dữ liệu kiểm thử, đồng thời cho phép người dùng phản hồi lại kết quả chấm điểm để cải thiện prompt và tiêu chí đánh giá. Ngoài ra, có thể tích hợp thêm email thật, lịch phỏng vấn, cổng ứng tuyển công khai và báo cáo phân tích hiệu quả tuyển dụng.

6.2.3 Về mở rộng nghiệp vụ

Ngoài phần kỹ thuật, hệ thống có thể được mở rộng thêm về mặt nghiệp vụ. Ví dụ, có thể bổ sung luồng tiếp nhận ứng viên từ cổng tuyển dụng công khai, quản lý lịch phỏng vấn, ghi nhận kết quả phỏng vấn và theo dõi các vòng tuyển dụng tiếp theo. Nếu phát triển theo hướng này, EasyHR sẽ tiến gần hơn tới một nền tảng hỗ trợ tuyển dụng tương đối trọn vẹn thay vì chỉ dừng ở bước sàng lọc ban đầu.

GHI CHÚ VỀ TÀI LIỆU THAM KHẢO

Các tài liệu tham khảo trong báo cáo chủ yếu là tài liệu chính thức của những công nghệ được sử dụng trong đề án, bao gồm FastAPI, React, PostgreSQL, Docker, Celery và Vite. Do EasyHR là một đề tài xây dựng hệ thống ứng dụng, báo cáo ưu tiên trích dẫn tài liệu kỹ thuật trực tiếp từ nhà phát triển công nghệ để bảo đảm tính chính xác.

Ngoài các tài liệu được liệt kê ở cuối báo cáo, nội dung mô tả nghiệp vụ và thiết kế hệ thống được tổng hợp từ quá trình phân tích, triển khai và kiểm thử trong chính dự án EasyHR.

TÀI LIỆU THAM KHẢO

- [1] FastAPI, *Fastapi documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [2] PostgreSQL Global Development Group, *Postgresql documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [3] Meta Open Source, *React documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://react.dev/>
- [4] Vite, *Vite documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://vite.dev/>
- [5] Celery Project, *Celery documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://docs.celeryq.dev/>
- [6] Docker Inc., *Docker documentation*, 2026. Accessed: May 23, 2026. [Online]. Available: <https://docs.docker.com/>

PHỤ LỤC

A. ĐẶC TẢ USE CASE CHÍNH

A.1 Use case tải lên và phân tích CV

Bảng A.1: Đặc tả use case tải lên CV

Tác nhân	Nhà tuyển dụng
Mục tiêu	Đưa một hoặc nhiều CV vào hệ thống và tạo hồ sơ ứng viên có cấu trúc.
Tiền điều kiện	Người dùng đã truy cập được hệ thống và có file PDF hợp lệ.
Luồng chính	Người dùng mở màn hình Candidates, chọn Upload resumes, chọn file PDF và xác nhận phân tích. Hệ thống kiểm tra định dạng, lưu file, trích xuất thông tin và tạo CandidateProfile.
Ngoại lệ	Nếu file không phải PDF hoặc quá trình phân tích lỗi, hệ thống hiển thị trạng thái failed và lý do lỗi nếu có.
Kết quả	CV được lưu trong ResumeDocument và thông tin ứng viên được lưu trong CandidateProfile.

A.2 Use case chấm điểm ứng viên

Bảng A.2: Đặc tả use case chấm điểm ứng viên

Tác nhân	Nhà tuyển dụng
Mục tiêu	Đánh giá mức độ phù hợp của ứng viên với một mô tả công việc.
Tiền điều kiện	Hệ thống đã có ít nhất một CandidateProfile và một JobDescription.
Luồng chính	Người dùng chọn mô tả công việc, chọn tập ứng viên, nhập ngưỡng điểm và trọng số tiêu chí. Hệ thống gửi dữ liệu cho service chấm điểm, nhận kết quả và lưu MatchRun cùng MatchResult.
Ngoại lệ	Nếu không có ứng viên, không có mô tả công việc hoặc trọng số không hợp lệ, hệ thống trả lỗi và yêu cầu người dùng chỉnh lại.
Kết quả	Người dùng xem được điểm tổng, điểm thành phần, trạng thái đạt ngưỡng và lý do đánh giá.

A.3 Use case tạo danh sách rút gọn**Bảng A.3:** Đặc tả use case tạo shortlist

Tác nhân	Nhà tuyển dụng
Mục tiêu	Lưu lại nhóm ứng viên tiềm năng để theo dõi tiếp.
Tiền điều kiện	Hệ thống đã có hồ sơ ứng viên và người dùng có quyền tạo collection.
Luồng chính	Người dùng tạo một shortlist mới, đặt tên và thêm ứng viên từ kết quả chấm điểm hoặc kết quả tìm kiếm.
Ngoại lệ	Nếu tên collection bị trùng trong phạm vi người tạo, hệ thống trả lỗi conflict. Nếu ứng viên đã tồn tại trong collection, hệ thống không thêm trùng.
Kết quả	ShortlistCollection và ShortlistItem được lưu trong cơ sở dữ liệu.

A.4 Use case chuẩn bị câu hỏi phỏng vấn**Bảng A.4:** Đặc tả use case chuẩn bị phỏng vấn

Tác nhân	Nhà tuyển dụng
Mục tiêu	Tạo hoặc lưu bộ câu hỏi phỏng vấn cho một ứng viên theo một vị trí cụ thể.
Tiền điều kiện	Có CandidateProfile và JobDescription tương ứng.
Luồng chính	Người dùng chọn ứng viên, chọn mô tả công việc và tạo bộ câu hỏi. Hệ thống lưu nội dung câu hỏi dưới dạng dữ liệu JSON để có thể hiển thị và chỉnh sửa.
Ngoại lệ	Nếu ứng viên hoặc mô tả công việc không tồn tại, hệ thống trả lỗi not found.
Kết quả	Bộ câu hỏi được lưu trong InterviewQuestion-Set.

B. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY THỬ

B.1 Yêu cầu môi trường

Để chạy thử hệ thống EasyHR trên máy cá nhân, cần cài Docker Desktop và Docker Compose. Nếu muốn sử dụng chức năng phân tích CV hoặc chấm điểm bằng LLM, cần cấu hình khóa API của nhà cung cấp mô hình trong biến môi trường.

B.2 Các bước chạy hệ thống

Từ thư mục gốc của dự án, chạy lệnh sau để khởi động các dịch vụ:

```
1 docker compose up --build -d
```

Sau khi container backend khởi động, chạy migration để tạo bảng cơ sở dữ liệu:

```
1 docker compose exec backend alembic -c alembic.ini upgrade head
```

Nếu cần tạo người dùng ban đầu và vai trò mặc định, chạy lệnh:

```
1 docker compose exec -T backend python seeds/seed_initial_user_and_roles.py
```

B.3 Địa chỉ truy cập

Sau khi chạy thành công, có thể truy cập giao diện frontend tại <http://localhost:5173>. Backend chạy tại <http://localhost:8000>. Tài liệu API tương tác của FastAPI có tại <http://localhost:8000/docs>.

B.4 Một số lỗi thường gặp

Nếu backend báo thiếu khóa API của LLM, cần kiểm tra lại biến môi trường như `GROQ_API_KEY`. Nếu API báo thiếu bảng trong cơ sở dữ liệu, cần chạy lại migration. Nếu upload CV lỗi do định dạng, cần kiểm tra file đầu vào có đúng định dạng PDF hay không.